



Test Automation Framework (TAF::Perl)

TAF Result Backend

MariaDB Foundation <https://mariadb.org>

Open, vendor-neutral, community-driven tooling for database testing and benchmarking.

TAF-Perl is maintained and published by the MariaDB Foundation to support transparent, reproducible, and fair performance evaluation across the open source database ecosystem. The framework is part of the Foundation's mission to provide open tooling that benefits the entire database community.

© MariaDB Foundation. All rights reserved.

TAF Results Backend Purpose and High Level Overview

1. What the backend results system is

The backend results system is the central storage and comparison engine for performance test results.

It receives structured output from TAF or any compatible runner, stores it in a durable and query-able form, and provides fast and consistent comparison logic so that regressions, improvements, and behavioral changes can be identified with confidence.

It is designed to be simple, predictable, and easy to maintain, offering a complete history of every run and every baseline decision.

2. What problem it solves

Performance testing produces large volumes of data, and without a structured backend it becomes difficult to track changes, identify regressions, or understand how results evolve over time.

The TAF backend results system solves this by providing:

- A consistent place to store all test runs
- A reliable way to identify workloads
- A fast and deterministic comparison mechanism
- A durable history of baselines and changes
- A clean interface for TAF and other tools to submit results

It eliminates the need for ad hoc scripts, manual comparisons, or scattered storage.

Everything is centralized, structured, and repeatable.

3. How it fits into the overall workflow

The backend results system sits between the test execution layer and any reporting or dashboard layer.

The workflow is:

- A test run completes in TAF or another runner.
- The Java parser receives the run output. Either the parser is passed either a raw results file and a database config file as arguments, or the raw results are streamed into the

parser through stdin while the database config is provided as a file. The parser supports both modes, allowing it to be used directly by TAF or by any external tool that pipes results into it.

- The parser prepares the data and sends it to the backend database using stored procedures.
- The backend stores the run, computes the workload identity, hashes it, and places the run in the correct bucket.
- If applicable, the backend selects the correct baseline, performs the comparison using a workload hash, and stores the results in the `test_run_comparison` table.
- The backend returns structured results consumable by dashboards, scripts, or automated systems.

This creates a clean separation between test execution, data ingestion, storage, and comparison.

4. What components exist

The backend results system comprises several parts that work together:

- Database
 - Stores all test runs, metrics, metadata, and baseline information.
- Tables
 - Define the structure of `test_run`, iteration metrics, workload identity, and supporting data.
- Stored procedures
 - Provide controlled entry points for inserting runs, updating baselines, and performing comparisons.
- SQL schema file
 - Contains the full definition of the backend database and is used to create or rebuild the system.
- Java parser
 - Reads raw test output, converts it into structured data, and calls the stored procedures. It handles JDBC, JSON parsing, and all communication with the backend.
- Configuration file
 - Defines the database connection details so the parser can talk to a local or remote backend.
- Working directory and source directory
 - The parser has a clear separation between source code and the working directory where logs, configs, and temporary files live.

Together, these components form a complete ingestion and comparison pipeline that is predictable, fast, and easy to maintain.

5. Deployment Models

The backend can be used in two different ways depending on how TAF is deployed and how results need to be stored or shared.

5.1 Centralized Backend (shared server)

In this model, a single backend database runs on a central server. All TAF installations, regardless of where they run, send their results to this shared backend.

This allows multiple test hosts, CI systems, or developers to contribute results to the same history and share baselines.

Characteristics:

- One backend database for all test hosts
- All TAF instances connect remotely using `backend.conf`
- Baselines are shared across all machines
- Ideal for team environments, CI pipelines, and long-term history

Requirements:

- The backend server must be reachable over the network
- `backend.conf` on each TAF host must point to the central server
- No special flags are required in test cases

5.2 Local Backend (per-host database)

In this model, each TAF installation runs its own backend database locally.

This is useful when tests must be isolated, when machines are offline, or when developers want a private results history.

Characteristics:

- Each TAF host has its own backend database
- Results are not shared between machines
- Baselines are local to each host
- Ideal for development, isolated testing, or offline environments

Requirements:

- Each local backend must have its own data directory, log directory, socket file, and port number that do not conflict with the database under test
- All test cases must include: `--ignore-running-db-process` so that TAF does not attempt to kill or interfere with the local backend database
- `backend.conf` must point to the local backend instance

Notes:

- The local backend must not share its port, socket, or directories with the database under test
- Each local backend must be initialized with its own schema and storage directories

TAF Backend Quick Start

This section provides the minimum steps required to set up the backend database, build the backend parser, and ingest a test run. It begins with the directory layout so you know exactly where everything is located.

1. TAF Backend Directory Layout (for orientation)

```
taf-perl/  
  libs/  
    reporter_libs/  
      backend/  
        build_backend_parser.sh  
        parse_raw_text_results.sh  
        backend_sql/create_backend.sql  
        backend_parser/
```

```
source/taf/backend/parser/  
  CliArgs.java  
  Db.java  
  DbConfig.java  
  DbIngest.java  
  JsonUtil.java  
  ParsedIteration.java  
  ParsedMetric.java  
  ParsedTestRun.java  
  Parser.java  
  PrimaryMetricClassifier.java  
  RawResultParser.java  
  RubyHashConverter.java  
  TafBackendCli.java  
  WorkloadDetector.java  
  WorkloadType.java  
working/  
  backend.conf.example  
  (jars go here)
```

2. Create the Backend Database

The documentation uses MariaDB for all examples.

Any database engine may be used as long as it supports the required schema and operates at the READ COMMITTED isolation level.

2.1 Ensure the database runs at READ COMMITTED

This backend requires READ COMMITTED isolation for correct ingest and comparison behavior.

For MariaDB, set in my.cnf:

```
[mysqld]  
transaction-isolation = READ-COMMITTED
```

Restart the server after editing.

2.2 Create the backend schema

Run the schema installer using the MariaDB client:

```
mariadb -u root -p < taf-perl/libs/reporter_libs/backend/backend_sql/create_backend.sql
```

This script:

- creates the taf_backend database
- creates all backend tables
- creates all views
- installs all stored procedures and functions
- initializes baseline comparison logic

All tables, views, and stored procedures are documented directly inside create_backend.sql and are also listed in the index section below.

3. Required Files (must exist before building the parser)

Place the following files in:

```
taf-perl/libs/reporter_libs/backend/backend_parser/working/
```

Required:

```
json-20260522.jar  
mariadb-java-client-3.5.8.jar
```

Copy the template config:

```
cp backend.conf.example backend.conf
```

A working JDK is required:

```
java -version  
javac -version
```

4. Build the Backend Parser

Run:

```
taf-perl/libs/reporter_libs/backend/build_backend_parser.sh
```

This script:

- removes old .class files
- compiles all Java sources under backend_parser/source
- uses json-20260522.jar on the classpath

- generates manifest.txt
- produces BackendParser.jar in backend_parser/working

5. Configure backend.conf

Edit:

backend_parser/working/backend.conf

Set:

```
db_host=127.0.0.1
db_port=3307
db_user=taf
db_pass=taf
db_name=taf_backend
```

6. Prepare a DB Config File

Example:

database_config_files/mysql/mysql_simple_2gbp.cnf

7. Ingest a Test Run

This section explains how to ingest a test run into the backend database. There are two supported ingestion paths:

1. Through TAF using backend_reporter.pm (automated)
2. Using raw results + stdin (manual or CI pipelines)

Both methods ultimately call the same Java parser and stored procedures.

7.1 Ingesting Through TAF (Automated)

When TAF completes a test run, it can automatically send the results to the backend. This is the recommended method for continuous testing, CI systems, and long-running performance environments.

Step 1 — Enable backend_reporter.pm

Edit your TAF configuration:

```
taf.report_plugin=taf_res_raw_text.pm,text.pm,...backend_reporter.pm
```

This ensures:

- raw results are generated
- HTML summaries are generated
- backend_reporter.pm is invoked last
- backend_reporter.pm calls the Java parser with the correct arguments

Step 2 — Ensure backend.conf is correct

backend_reporter.pm reads:

```
backend_parser/working/backend.conf
```

This file must contain:

```
db_host=127.0.0.1
db_port=3307
db_user=taf
db_pass=taf
db_name=taf_backend
```

TAF will pass this file to the parser automatically.

Step 3 — Run a TAF test

When the test finishes, TAF will:

- generate raw results
- call the Java parser
- ingest the run into the backend
- print a summary of the ingest result

Step 4 — Verify ingest

Run:

```
SELECT id, test_name, test_timestamp
FROM test_run
ORDER BY id DESC
LIMIT 5;
```

If the run appears, ingestion succeeded.

7.2 Ingesting Using Raw Results + stdin (Manual or CI)

This method is useful when:

- running sysbench or custom workloads outside TAF
- ingesting historical results

- debugging parser behavior
- using CI systems that produce raw output

Step 1 — Produce a raw results file

Example:

```
sysbench oltp_read_write.lua ... > raw_results.txt
```

Step 2 — Prepare a DB config file

Example:

```
database_config_files/mysql/mysql_simple_2gbp.cnf
```

This file describes the database under test (not the backend DB).

Step 3 — Run the parser using stdin

```
cat raw_results.txt | \
java -jar backend_parser/working/BackendParser.jar \
  --stdin \
  --db-config-file mysql_simple_2gbp.cnf \
  --backend-config backend_parser/working/backend.conf
```

The parser will:

- read raw results from stdin
- parse all metrics, iterations, and metadata
- compute workload identity
- insert system identity
- insert test_run
- insert all metrics
- compute workload hash
- check for baseline
- perform comparison if applicable

Step 4 — Verify ingest

```
SELECT id, test_name, test_timestamp
FROM test_run
ORDER BY id DESC
LIMIT 5;
```

7.3 What Happens During Ingest (Internal Flow)

Regardless of ingestion method, the backend performs the same steps:

1. Parse raw results

Extract test name, workload type, thread counts, iterations, metrics, metadata.

2. Insert system identity

CPU, RAM, OS, kernel, host hash.

3. Insert DB config identity

DB version, engine, settings, connector.

4. Insert test_run

Full identity record for the run.

5. Insert metrics

One row per metric per iteration per thread.

6. Compute workload hash

Groups runs with identical workload identity.

7. Baseline selection

If no baseline exists → this run becomes baseline.

If a baseline exists → comparison is performed.

8. Comparison result stored

test_run_comparison row created if applicable.

This guarantees deterministic, repeatable ingest behavior.

7.4 Common Ingest Errors

Missing backend.conf

Parser fails with connection errors.

Wrong DB port

Local backend must not conflict with the DB under test.

Incorrect DB config file

Parser cannot determine workload identity.

Raw results truncated

Parser will reject malformed or incomplete input.

7.5. Verify Ingest

```
SELECT id, test_name, test_timestamp FROM test_run ORDER BY id DESC LIMIT 5;
```

Database Layer

1. Overview

The database is the core of the backend results system. It stores every test run, every metric, every piece of metadata, and every baseline designation. All comparisons, lookups, and history tracking depend on the structure and relationships defined here. The database is intentionally simple, normalized, and predictable so that it can scale cleanly and remain easy to maintain.

The database layout is MariaDB-based, and the entire schema (tables, indexes, stored procedures) is created from a single SQL file included with the backend.

2. Tables and Their Roles

The backend currently contains the following tables:

- db_config_param
- db_config_set
- results
- system_under_test
- test_run
- test_run_comparison

db_config_set

- Defines a single configuration set used for a test run.

Purpose:

- Store the identity of a config file. If the file contents are identical, the hash matches and the same config_set row is reused.

db_config_param

- Stores individual parameters extracted from a config file.

Purpose:

- Provide full transparency into the exact settings used for a run.

system_under_test

- Describes the exact machine where the test was executed.

Purpose:

- Record the hardware and OS environment for which the results belong. The `system_hash` uniquely identifies the host so the backend can detect when a run came from a different machine.

test_run

- Represents a single execution of a workload.
- This is the central, most detailed table in the backend.

Purpose:

- Capture the full identity of a test run — workload, harness, DB version, environment, parameters, thresholds, metadata, and timing.

Major field groups:

- Workload identity
- Harness identity
- Client identity
- Database identity
- Configuration parameters
- Thresholds
- Baseline flags
- Timing
- Metadata (JSON)
- System link (`system_id`)

This table is intentionally exhaustive so any run can be reconstructed or audited years later.

results

- Stores all numeric and time-series metrics produced during a test run.
- This table is the meat of the performance data — every throughput number, latency statistic, count, and derived metric ends up here.

Purpose:

- Record every metric produced by a test run, across all thread counts and iterations, in a structured and queryable form.

Each row represents:

- one metric
- for one test_run
- at one thread count
- for one iteration
- over one run_time window

Key fields:

- test_run_id — links the metric to the specific run
- threads — thread count used
- iteration — iteration number
- run_time — duration of the measurement window
- metric_name — e.g., TPS, latency_avg, total_queries
- metric_type — primary or additional
- metric_dimension — throughput, time, count, etc.
- metric_unit — tps, qps, ms, count
- metric_value — numeric value
- variable_description / variable_value — optional
- extra_data — optional JSON/text
- last_modified — insertion timestamp

Example metrics captured:

- TPS
- total_read_ops
- total_write_ops
- total_queries
- queries_per_second
- events_ps
- latency_min / avg / max / 95th percentile
- latency_sum
- thread_event_avg / stddev
- threads_execution_avg / stddev

Note on metric_type:

- Only one metric per test_run / thread / iteration may be marked primary.
- All others for that iteration are additional.
- This allows frontends to know which metric to use as the main performance value when rendering results.

test_run_comparison

- Stores the summary result of comparing a baseline test run against a candidate test run.

Purpose:

- Record the outcome of a comparison between two runs, including the direction of change, the magnitude of change, and the specific thread where the largest difference occurred. This table provides a durable, queryable summary of whether a new run improved, regressed, or remained within tolerance.

Key fields:

- baseline_run_id — test_run ID of the baseline
- candidate_run_id — test_run ID of the candidate
- status — GAIN, REGRESSED, WARNING, PAR, PAR+, PAR -
- thread_of_max_change — thread count where the largest change occurred
- max_change_pct — percentage change at that thread
- max_change_value — raw numeric difference

- `baseline_value_at_max` — baseline value at that thread
- `candidate_value_at_max` — candidate value at that thread
- `comparison_hash` — SHA-256 identity of the comparison
- `created_at` — timestamp

Example row:

```

baseline_run_id:    2
candidate_run_id:   3
status:             GAIN
thread_of_max_change: 4
max_change_pct:     8.422791063569617
max_change_value:   1912.77500000000015
baseline_value_at_max: 22709.515
candidate_value_at_max: 24622.29
comparison_hash:
8e0375adfc1f456327247b91b11520cf9c2616e04503a43a71873485a07a6542
created_at:         2026-06-05 16:39:07

```

3. Stored Procedures Overview

The backend defines the following stored procedures. Each procedure has a single, clear responsibility and all are documented in `create_backend.sql`.

check_for_baseline

Determines whether a newly ingested test run should become the baseline for its workload hash. If a baseline exists, it triggers comparison logic.

do_compare

Performs the metric-level comparison between a baseline and a candidate run and inserts a summary row into `test_run_comparison`.

get_candidate_baselines

Returns all valid baseline candidates for a given test run based on workload hash and timestamp ordering.

insert_full_db_config

Inserts a complete database configuration record for reproducibility. This captures DB version, engine, settings, and connector details.

insert_result

Inserts a single metric result row for a specific iteration and thread. This is called repeatedly during ingest to populate iteration_metric.

insert_system_identity

Inserts or updates the system identity information associated with a test run, including CPU, RAM, OS, kernel, and host metadata.

insert_test_run

Inserts the main test_run record including workload identity, metadata, timestamps, and configuration references. This is the entry point for a new run.

report_comparison_full

Returns a full comparison report including baseline metadata, candidate metadata, TPS summary, thread deltas, and raw metric summaries.

report_full_from_test_run_comparison_id

Same as report_comparison_full but looks up the comparison by its comparison_id instead of requiring both run IDs.

set_baseline

Marks a specific test run as the baseline for its workload hash. Used when manually promoting a run to baseline.

set_baseline_with_comment

Same as `set_baseline` but allows attaching a human-readable comment explaining why the baseline was chosen.

taf_report

A human-readable reporting helper. Returns a formatted summary of a single test run, including system info, DB info, thread summaries, and all metrics.

It is not used by TAF ingestion or comparison logic; it exists only as a convenience procedure for interactive inspection of stored results.

update_workload_hash

Computes and updates the `workload_hash` for a test run based on its identity fields. This ensures that runs with identical workload characteristics are grouped together.

4. Views Overview

The backend defines three views. These views provide simplified, read-only access to the most important backend data structures. Each view is documented in `create_backend.sql`.

baseline_history_view

Shows the full history of baseline selections for every workload hash. This includes which test runs were chosen as baselines, when they were chosen, and any associated comments.

comparison_summary_view

Provides a summary of all comparisons performed by the backend. Each row represents a baseline/candidate pair along with the comparison status (improved, regressed, or neutral) and key metadata.

workload_baseline_latest_view

Returns the latest baseline for each workload hash. This is used by TAF and external tools to quickly determine which baseline is currently active for a

given workload.

Baseline Management

This section explains how baselines are set, stored, selected, and used by the backend. Baselines are central to the comparison pipeline: they define the “known good” performance for a workload so that regressions and improvements can be detected reliably.

Baseline management in the backend is deterministic, fully automated, and history-preserving. No baseline is ever deleted or overwritten; every baseline decision is stored permanently.

1. What a Baseline Is

A baseline is a specific `test_run` that represents the current “known good” performance for a workload. All future runs of the same workload are compared against this baseline.

A baseline is defined by:

- workload identity (test name, DB version, config, system identity, etc.)
- workload_hash (computed from identity fields)
- is_baseline flag in test_run
- baseline_set_at timestamp
- optional baseline comment (if manually set)

Only one baseline is active per workload_hash at any given time, but all historical baselines are preserved.

2. How Baselines Are Selected For Comparison

When a new `test_run` is ingested, the backend executes:

```
CALL check_for_baseline(p_run_id);
```

This stored procedure performs the following steps:

1. Look up all existing baselines for that workload_hash.
2. If no baseline exists:
 - Check is over.
3. If a baseline exists:
 - The new run is compared against the existing baseline.
 - The comparison result is stored in test_run_comparison.

3. How Baselines Are Set

A user may promote any test_run to baseline using: `CALL set_baseline(p_run_id);`
or, with a comment: `CALL set_baseline_with_comment(p_run_id, 'reason for promotion');`

Manual promotion:

- sets is_baseline = TRUE
- updates baseline_set_at
- stores the comment (if provided)
- does NOT delete or modify previous baselines
- does NOT require comparison

This allows controlled baseline updates when a new release, configuration, or hardware change is introduced.

4. How Baselines Are Stored

Baselines are stored directly in the test_run table:

- is_baseline → TRUE for baseline runs
- baseline_set_at → timestamp of promotion
- baseline_comment → optional text
- workload_hash → groups runs by workload identity

Because baselines live inside test_run, the backend preserves:

- full history of all baselines

- full history of all comparisons
- complete lineage of performance evolution

No baseline is ever removed or overwritten.

5. How Baselines Are Used During Comparison

When a new run is ingested and a baseline exists:

- `do_compare()` is executed
- metric-level differences are computed
- the largest change is identified
- status is assigned (GAIN, REGRESSED, PAR, PAR+, PAR-)
- a row is inserted into `test_run_comparison`

This comparison is deterministic and based on:

- primary metric only
- `thread_of_max_change`
- `max_change_pct`
- `max_change_value`

Dashboards and reporting tools read from:

- `comparison_summary_view`
- `workload_baseline_latest_view`

to show the current baseline and the latest comparison results.

6. How Baseline History Is Tracked

The backend provides:

`baseline_history_view`

This view shows:

- every baseline ever set
- when it was set
- who set it (if tracked externally)

- any comments
- workload_hash lineage

This allows long-term tracking of performance evolution across releases, hardware changes, and configuration updates.

7. Why Baselines Are Never Deleted

Baselines represent historical truth. Removing or overwriting them would:

- break comparison history
- destroy lineage
- make regressions impossible to track
- invalidate dashboards and reports

By storing baselines directly in test_run and never deleting them, the backend guarantees:

- reproducibility
- audibility
- long-term stability
- deterministic comparison behavior

Workload Hash Model

The workload hash is the core identity mechanism of the backend. It ensures that test runs with identical workload characteristics are grouped together, compared consistently, and assigned the correct baseline. The workload hash is computed exactly as implemented in `computeWorkloadHash(ParsedTestRun tr)`, using a canonical identity string and SHA-256. Only workload-defining inputs are included. All performance outputs are excluded.

1. Purpose of the Workload Hash

The workload hash solves three critical problems:

- Grouping identical workloads into the same comparison bucket.
- Preventing comparisons between runs that differ in workload-defining inputs.
- Ensuring deterministic baseline selection per workload_hash.

Baselines and comparisons are always keyed by workload_hash.

2. Exact Fields Included in the Workload Hash

The canonical identity string is constructed in this exact order:

Test identity

TEST=<testName>

SUITE=<suiteName>

SUITE_VER=<suiteVersion>

SUITE_REV=<testSuiteRevision>

System identity

SYS=<systemId>

Database identity

DB_MAKER=<dbMaker>

DB_ENGINE=<dbEngine>

DB_VER=<major version only>

DB_VER is normalized by stripping all non-numeric suffixes and taking only the major component: e.g., "12.3.2-MariaDB" → "12"

Config identity

CFG=<configId>

Workload parameters

THREADS=<threadList>

WARM_THREADS=<warmupThreads>

WARM_DURATION=<warmupDuration>

ROWS=<numberOfRows>

TABLES=<numberOfTables>

RANGE=<rangeSize>

CONNECTOR=<connector>

Versions

TAF_VER=<tafHarnessVersion>

CLIENT_VER=<clientVersion>

Iteration structure

ITER=<number of iterations>

Requested test duration

DURATION=<testDuration>

3. What Is Explicitly Excluded

The workload hash does NOT include:

- TPS, QPS, latency, or any metric values
- timestamps
- actual run_time
- baseline flags
- comparison results
- comments
- iteration-level measurements
- any performance output

These are outputs and must not affect workload identity.

4. How the Backend Uses the Workload Hash

The workload hash is used in:

Baseline selection

- If no baseline exists → no compare done.
- If a baseline exists → new run is compared against it.

Comparison pipeline

- `do_compare()` only compares runs with the same `workload_hash`.

Views and reporting

- `workload_baseline_latest_view`
- `baseline_history_view`
- `comparison_summary_view`

History tracking

- All baseline and comparison lineage is grouped by `workload_hash`.

Preventing invalid comparisons

- If any identity field differs, the hash differs, and the backend will not compare the runs.

Summary

The workload hash:

- Uniquely identifies a workload using only input parameters.
- Includes all identity fields defined in `computeWorkloadHash()`.
- Excludes all performance outputs.
- Guarantees deterministic baseline and comparison behavior.
- Ensures long-term, reliable regression tracking.

It is the foundation that makes the backend safe, predictable, and trustworthy for performance analysis.